

研究生项目教材

AI-assisted LQCD Meson DA Analysis Agent

系统设计与训练方案

项目导师版 / 学生执行版

2026 年 5 月 17 日

如何使用本教材

本教材按照 **Part** → **Chapter** → **实战任务** 的层次组织，适合研究生在一个学期（三个月）内边学边做。

- **Part I–II**: 建立项目全局观和核心理念——**前两周读完**。
- **Part III–IV**: 理解 agent 角色设计和技术栈选型——**第一个月内反复参考**。
- **Part V**: 三个月实战课程的核心，每周有具体任务、命令、输出文件和验收标准——**每周跟着做**。
- **Part VI**: 10 个 Codex 练习，穿插在 Part V 的周任务中——**按需使用**。
- **Part VII**: 最终验收和未来升级路线——**Months 2–3 开始读**。

每个 Chapter 以目标说明开头，结尾有**本章交付清单**。Part V 中每周都有**学习目标、具体任务、Codex prompt、命令示例、输出文件列表、pytest 验收、导师检查问题和常见错误**。

配套详细文档：

- `lqcd_agent_month1_detailed_guide_CN.pdf`: Month 1 物理背景、bootstrap/ χ^2 推导、逐日任务单
- `lqcd_agent_month2_detailed_guide_CN.pdf`: Snakemake rule 详细写法、agent rule 实现、每周任务
- `lqcd_agent_month3_detailed_guide_CN.pdf`: LangGraph state/graph、approval packet、evaluation harness

目录

第一部分 项目总览	6
1 项目目标与最终产品形态	6
1.1 本项目不是什么	6
1.2 本项目是什么	6
1.3 三个月后产品应展示的能力	6
1.4 贯穿全文的真实 Example	7
2 学习目标与技能分类	8
2.1 LQCD Analysis 学习目标	8
2.2 Software Engineering / Reproducibility 学习目标	8
2.3 AI Agent System 学习目标	9
2.4 如何使用本教材的三个阅读循环	9
第二部分 核心理念	10
3 Risk-Based Autonomy: 从全手动到研究驾驶舱	10
3.1 Autonomy Level: AI 自主程度的五个台阶	10
3.2 四级风险与对应的人类介入规则	10
3.3 AI 决策边界: 三类权限的明确划分	11
3.4 AI 决策边界对照表	12
3.5 Human Approval Packet 文件结构	12
3.6 典型场景: 四种风险等级的处理流程	13
4 从 TradingAgents 到 LQCD Research Group Agents	15
4.1 借鉴什么: TradingAgents 的核心设计思想	15
4.2 角色映射表: 从金融 Agent 到 LQCD Analysis Agents	15
4.3 为什么不用金融名字?	16
第三部分 Agent 角色设计	17
5 LQCD Research Group 风格的 Multi-Agent 架构	17
5.1 设计原则: 像一个真实的 LQCD 分析子组	17
5.2 Agent 体系总览 (五层架构)	17
5.3 统一 Agent 介绍模板	18
5.4 Agent 1: Analysis Engineer Agent (L1: 工程与执行)	19
5.5 Agent 2: Correlator Analyst Agent (L2: 数据与拟合分析)	20

5.6	Agent 3: Fit Strategy Postdoc Agent (L2: 数据与拟合分析)	21
5.7	Agent 4: Matrix Element Analyst Agent (L2: 数据与拟合分析)	22
5.8	Agent 5: Critical Referee Agent (L4: 批判与决策)	23
5.9	Agent 6: Analysis Coordinator / PI Agent (L4: 批判与决策)	24
5.10	Agent 7: Human Approval Gate (L4: Risk-Based Conditional Gate)	25
5.11	Agent 8: Analysis Memory Curator (L5: 长期学习)	26
5.12	Agent 9 & 10: Lattice QCD Physics Expert + Perturbative Matching Expert (L3: 高级物理审查)	27
6	Agent 工作流: 从数据到决策	29
6.1	第一阶段: 真实数据分析 pipeline (Month 1-2 实现)	29
6.2	第二阶段: 加入理论 matching (Month 3+)	29
6.3	第三阶段: 多 Agent Debate (Advanced roadmap)	29
	 第四部分 技术栈地图	 31
7	工具、职责与选型理由	31
7.1	Reproducible Analysis: config、schema 与结构化输出	31
7.2	Testing: pytest	31
7.3	Scientific Workflow: Snakemake	31
7.4	Agent Orchestration: LangGraph (或 Python State Machine)	32
7.5	Memory: 从 YAML 到 SQLite/DuckDB/RAG	32
7.6	Observability: run.log、decision_log.json 与未来 tracing	33
7.7	AI-Assisted Development: Codex	33
7.8	技术栈全景图	33
	 第五部分 三个月实战课程	 35
8	Month 1: 从 Golden Example 到可复现 CLI	36
8.1	Month 1 目标	36
8.2	Week 1: 环境、Golden Example 复现与 Data-Flow Map	36
8.3	Week 2: Config-Driven CLI + Pydantic Schema	37
8.4	Week 3: Correlator Diagnostics + pytest	38
8.5	Week 4: Month 1 Integration Demo	39
9	Month 2: Workflow、Postdoc-level Diagnostics 与 Decision Log	41
9.1	Month 2 目标	41
9.2	Week 5: Snakemake Workflow	41
9.3	Week 6: Fit Strategy Postdoc Agent	41
9.4	Week 7: Matrix Element Analyst + Critical Referee	42

9.5 Week 8: Analysis Coordinator + Decision Log + Month 2 Demo	42
10 Month 3: Multi-Agent Product Demo	44
10.1 Month 3 目标	44
10.2 Week 9: Python State Machine / LangGraph Prototype	44
10.3 Week 10: Human Approval Gate + Case Memory	45
10.4 Week 11: Evaluation Harness + Final Report 自动生成	45
10.5 Week 12: Final Demo + Documentation + Productization	46
第六部分 Codex 实战练习	48
11 10 个 Codex Prompt 练习	48
11.1 练习 1: 生成 Pydantic Config Schema	48
11.2 练习 2: 写 Python CLI	48
11.3 练习 3: 写 pytest	49
11.4 练习 4: 用 Traceback 修 Bug	49
11.5 练习 5: 写 Snakemake Rule	50
11.6 练习 6: 写 Fit Diagnostics	50
11.7 练习 7: 写 Critical Referee Objection Logic	51
11.8 练习 8: 写 Approval Packet Generator	52
11.9 练习 9: 写 Memory Update Function	53
11.10 练习 10: 写 Final Report Template	53
11.11 Codex 使用黄金法则	54
第七部分 最终验收与升级路线	56
12 Final Demo 验收标准	56
12.1 最终产品必须展示的能力	56
12.2 导师验收问题清单	56
12.3 每月汇报模板	56
12.4 导师检查清单（三个月汇总）	57
12.5 最终交付清单	57
13 未来升级方向：如何变成更高级的系统	59
13.1 升级路线图总览	59
13.2 升级 1: 从 Rule-Based 到 LLM-Assisted Analysis Coordinator	59
13.3 升级 2: 从 Single Referee 到 Multi-Agent Debate	60
13.4 升级 3: 从 YAML Memory 到 SQLite/DuckDB + RAG	60
13.5 升级 4: 从 Single Case 到 Ensemble-Level Campaign	60
13.6 升级 5 & 6: 加入 LQCD Physics Expert 和 Perturbative Matching Expert	60

13.7 升级 7 & 8: Observability + Interactive Review	61
13.8 最终理想形态: LQCD Analysis Subgroup as Code	61
附录 A: 推荐最终 Repo 结构	62
附录 B: 参考资料与工具选择依据	63

第一部分 项目总览

1 项目目标与最终产品形态

1.1 本项目不是什么

- 不是普通脚本集合。不是让学生把几个 notebook 跑通就交差。
- 不是简单聊天机器人。不是套一个 LLM 对话界面就叫“科研 AI agent”。
- 不是手动流程的自动化 wrapper。不是只把人工操作换成 shell script。

1.2 本项目是什么

本项目是一个 **可复现、可审计、可逐步自主决策的 LQCD meson DA analysis agent system**。目标是把真实的格点 QCD meson DA 数据分析流程尽可能交给 AI agents 自动执行、自动检查、自动复现和自动记录，只在关键决策、风险升级和最终物理结论时才引入人工判断。

系统的五个核心设计目标：

1. **可复现**：所有分析由 config、schema、workflow、日志和版本控制驱动，而不是依赖手动 notebook 操作。
2. **可追责**：每一个重要判断都要留下 decision log，包括输入、候选方案、选择理由、风险等级和是否需要人工批准。
3. **AI-first automation**：优先让 AI 自动完成重复性、结构化、可检查的科研工程工作。
4. **Risk-based human-gated**：按风险等级决定人类介入深度。Low risk 自动推进，Medium risk 推荐复核，High risk 必须审批，Critical risk 禁止 AI 自行推进。
5. **真实科研可用**：最终应能让 AI 尽可能承担真实 meson DA 分析中的常规工作，而不是停留在玩具示例。

1.3 三个月后产品应展示的能力

三个月结束时，原型系统应能对一个真实 golden example case 展示以下闭环：

1. 读取一个 golden example case 的 config 和数据；
2. 自动完成 data check、analysis run、fit diagnostics、matrix element checks；
3. 自动生成 plots、summary.csv、result.json；
4. 多 agent 给出 recommendation 和 objection；
5. Analysis Coordinator 生成 decision_log.json；
6. 低风险自动推进并写入 memory，中高风险生成 approval packet 并触发 Human Approval Gate；
7. case memory 记录 accepted / rejected / failed cases；

8. 最终生成一份 human-readable analysis report。

1.4 贯穿全文的真实 Example

本项目以仓库中的 `examples/l64c64a076_m140/` 作为贯穿全文的真实 example。这不是 toy data，而是一套已经裁剪成 DA-only 目标的实际分析链。

当前 example 的真实分析顺序：

阶段	结果层级	关键问题
0-effective-mass	effective mass 和图	plateau、噪声、bootstrap 误差是否合理
1-c2pt-fit	两点拟合	能量、overlap、 χ^2/dof 是否稳定
2-bm	DA bare matrix element	是否已把 ratio 中的矩阵元提取出来
3-normalize	归一化后的中间量	只保留 mode3 路径，检查归一化是否合理
4-FT	Fourier 变换结果	最终分布的形状和物理趋势是否正常

这条链的关键约束：只保留 `b_X`、`OT5`、`bT=0`；删除 `b_Y`、`OZ5`；下游只保留 `mode3` 路径。

本章交付清单：

- 能说出本项目的五个核心设计目标；
- 能画出 golden example 的分析链（5 个阶段）；
- 能说出三个月后原型系统的 8 项能力。

2 学习目标与技能分类

本章明确学生三个月的学习目标，按三类技能组织。学生不需要一开始掌握所有内容，但需要在三个月内逐步达到熟练水平。

2.1 LQCD Analysis 学习目标

需要掌握：

- 理解 2pt correlator 的基本结构和 effective mass 的含义；
- 理解 bootstrap / jackknife 的统计原理，能解释为什么需要 binning；
- 理解 χ^2/dof 在拟合质量判断中的作用和局限；
- 理解 1-state / 2-state / 3-state fit 的物理含义和选择依据；
- 理解 fit window ($t_{\min}$ / $t_{\max}$) 和 prior 如何影响拟合结果；
- 理解 DA ratio 的基本结构，知道 bare matrix element 如何从 ratio 中提取；
- 理解 z -dependence 和 P_z -dependence 的物理意义；
- 理解 real/imaginary symmetry 和 normalization condition 的含义。

不要求一开始掌握：

- 详细的 LQCD 场论推导；
- LaMET / factorization / matching 的完整理论；
- 多个 ensemble 的 continuum extrapolation；
- 完整的 systematic uncertainty budget 估计。

2.2 Software Engineering / Reproducibility 学习目标

需要掌握：

- 把 notebook-style 分析改造成 config-driven Python CLI；
- 用 Pydantic 定义输入输出 schema；
- 用 pytest 写测试并验收 Codex 生成的代码；
- 用 Snakemake 管理文件依赖和增量重跑；
- 用 Git 做小步提交（每完成一个模块就 commit）；
- 写 manifest、run.log、result.json 以记录 provenance；
- 理解 overwrite protection 和 output validation 的重要性。

不要求一开始掌握：

- Docker / containerization；

- CI/CD pipeline;
- 复杂的 HPC 调度 (SLURM 等);
- 大规模数据管理。

2.3 AI Agent System 学习目标

需要掌握:

- 理解 multi-agent division of labor: 每个 agent 有明确职责和边界;
- 理解 Risk-Based Autonomy: 什么风险等级对应什么自动化程度;
- 理解 debate/critic 机制: Critical Referee 如何寻找漏洞;
- 理解 decision log 的格式和可追责性要求;
- 理解 Human Approval Gate 的触发条件和审批包结构;
- 理解 case memory 的结构化存储和检索需求;
- 用 LangGraph 或 Python state machine 表达 agent 状态流;
- 写 Codex prompt 让 AI 生成代码、测试和文档, 并尽量减少人类手工写实现。

不要求一开始掌握:

- LLM fine-tuning 或 RLHF;
- 复杂的 RAG pipeline;
- 大规模 agent 系统的运维和监控;
- 多用户协作的 agent 系统设计。

2.4 如何使用本教材的三个阅读循环

1. **第一遍 (Month 1 Week 1):** 通读 Part I-II (Ch.1-4), 建立项目全局观。然后进入 Part V Ch.8, 开始 Month 1 实战。
2. **第二遍 (Month 2):** 精读 Part III (Ch.5-6) 的 agent 定义, 深入理解每个 agent 的输入输出和职责边界。参考 Part IV (Ch.7) 的技术栈细节。
3. **第三遍 (Month 3):** 重读 Part II (Ch.3) 的 Risk-Based Autonomy, 对照 Part VII (Ch.12-13) 的验收标准和未来方向, 确保系统设计方向正确。

本章交付清单:

- 能列出三类学习目标中自己最需要加强的 3-5 项;
- 能说出三个阅读循环分别对应哪个训练阶段。

第二部分 核心理念

3 Risk-Based Autonomy: 从全手动到研究驾驶舱

3.1 Autonomy Level: AI 自主程度的五个台阶

本项目的长期目标不是停留在”AI 帮人跑脚本”，而是让 AI agents 逐步承担越来越多的 analysis decisions。下表定义五个自主等级：

等级	名称	含义
A0	Manual	人类手动完成所有分析步骤；AI 最多提供代码补全。
A1	AI-assisted coding	人类用 Codex 生成代码，但仍由人类检查、运行、解释所有结果。
A2	Semi-autonomous workflow	Snakemake 管理依赖；Analysis Engineer 自动执行 workflow；结构化输出自动生成。人类审查关键结果。
A3	Risk-based autonomous system	多 agent 分工协作；低风险自动推进，中风险推荐复核，高风险生成审批包。人类只在关键节点介入。
A4	Research copilot group	AI agents 像一个 LQCD analysis subgroup 一样协同工作；自动承担大部分分析决策；人类主要提出物理目标、审查高风险结论、承担最终责任。

本项目的前三个月目标是达到 **A2–A3 的初级版本**（A2 在 Month 2 完成，A3 在 Month 3 完成）。长期目标接近 A4。

3.2 四级风险与对应的人类介入规则

所有 AI decision 都按风险等级分级处理。Human Approval Gate 不是每一步都要人类审批，而是基于风险的、条件触发的 exception path。

风险等级	AI 行为	结果状态	人类介入
Low	自动执行、自动记录、自动推进 workflow	自动进入 accepted candidate	不需要
Medium	继续推进 workflow, 标记 human_review_recommended=true	pending candidate, 不能自动升级	建议 复核（非强制）
High	生成 approval_packet, workflow 在 promotion 前暂停	pending approval	必须 Human Approval Gate 审批
Critical	AI 不允许 继续推进科学结论	系统暂停，等待裁决	必须 Human PI / advisor 亲自决定

3.3 AI 决策边界：三类权限的明确划分

AI 可以自动决定（无需人类介入）：

- 检测 missing files 并报错
- config validation failure 的诊断和提示
- routine rerun（参数未变的重复执行）
- failed fit 的自动标记和记录
- bad candidate fit 的自动拒绝（基于明确的 χ^2/dof 或稳定性阈值）
- diagnostic plot 的生成和归档
- targeted rerun suggestion 的提出
- low-risk workflow 的自动推进和 memory update

AI 可以推荐但不能自动最终批准：

- central fit candidate 的选择
- systematic variation set 的定义
- data exclusion 的建议（标记 exclusion candidate）
- accepted candidate 的 promotion（标记 pending promotion）
- physics interpretation 的初步撰写

必须 Human Approval Gate（不可绕过）：

- final central value 的确定
- final systematic uncertainty 的确定

- final accepted result 的 promotion (pending $\rightarrow$ accepted)
- final data exclusion 的确认
- paper-level claim 的审批
- major analysis strategy change 的批准
- overriding Critical Referee 的 major objection

3.4 AI 决策边界对照表

决策类型	AI 自动决定	AI 推荐	必须人工
missing files / config validation failure	✓	—	—
routine rerun	✓	—	—
failed fit rejection	✓	—	—
bad candidate fit rejection	✓	—	—
diagnostic plot generation	✓	—	—
targeted rerun suggestion	✓	—	—
low-risk workflow continuation	✓	—	—
central fit candidate selection	—	✓	—
systematic variation set	—	✓	—
data exclusion proposal	—	✓	—
candidate promotion to accepted	—	✓	—
physics interpretation draft	—	✓	—
final central value	—	—	✓
final systematic uncertainty	—	—	✓
final accepted result promotion	—	—	✓
final data exclusion	—	—	✓
paper-level claim	—	—	✓
major strategy change	—	—	✓
override Critical Referee major objection	—	—	✓

3.5 Human Approval Packet 文件结构

当 Analysis Coordinator 判定为 High risk 时，系统生成以下审批包：

```

outputs/<case_id>/approval_packet/
  00_requested_decision.md          # 需要人类做哪个决定
  01_summary_for_human.md          # 人类可读的摘要 (含关键数字和图表引用)
  02_key_plots.pdf                  # 关键诊断图 (effective mass, fit stability 等)
  03_fit_comparison.csv             # 候选 fit 对比表
  04_critical_referee_report.json   # Critical Referee 完整报告
  05_ai_recommendation.json         # AI 推荐及理由
  06_decision_options.yaml          # 可选决策项及后果
  07_human_decision.yaml            # 人类填写后返回 (系统读取)

```

06_decision_options.yaml 示例:

```

decision_request:
  case_id: "l64c64a076_m140"
  risk_level: "high"
  question: "Select central fit candidate"
  ai_recommendation:
    preferred: "2-state fit (tmin=5)"
    reason: "Better chi2 stability across the scan window"
  options:
    - id: "option_a"
      label: "2-state fit (tmin=5) as central"
      risk_if_chosen: "medium"
      ai_preference: "recommended"
    - id: "option_b"
      label: "1-state fit (tmin=8) as central"
      risk_if_chosen: "medium"
      ai_preference: "alternative"
  deadline: "2026-06-01"
  fallback_if_no_response: "hold_and_remind"

```

07_human_decision.yaml 由人类填写:

```

human_decision:
  case_id: "l64c64a076_m140"
  chosen_option: "option_a"
  approver: "advisor_name"
  comments: "Accepted 2-state as central. Add 1-state as systematic in next round."
  timestamp: "2026-05-20T15:00:00"

```

3.6 典型场景：四种风险等级的处理流程

1. **Good case (Low risk):** Correlator Analyst 报告 plateau 稳定, Fit Strategy Postdoc 推荐 2-state fit, Critical Referee 无 major objection. Analysis Coordinator 判定 low risk, 自动推进, 写 decision_log, 更新 accepted memory. 人类不需要介入。

2. **Ambiguous case (Medium risk):** 1-state 和 2-state 结果相差 1.8 sigma, 但各自的 fit 质量可接受。Analysis Coordinator 判定 medium risk, 标记 pending candidate, 推荐人类看一下。Workflow 不阻塞, 结果暂时不升级为 final。
3. **High-stakes case (High risk):** Critical Referee 提出 major objection: P_z dependence 在归一化后改变趋势。Analysis Coordinator 判定 high risk, 生成 approval packet, workflow 在 promotion 前暂停。人类导师审批后决定: 接受 2-state 为 central, 并要求下一轮检查 P_z systematics。
4. **Crisis case (Critical risk):** Critical Referee 发现 local matrix element 违反 normalization condition, 且 Fit Strategy Postdoc A 和 B 的基础假设互相矛盾。Analysis Coordinator 判定 critical risk, 系统暂停, 通知 PI。PI 组织人工 review 后决定方案。

本章交付清单:

- 能解释五个 Autonomy Level 的含义和前三个月的目标等级;
- 能默写出四种风险等级对应的 AI 行为和人类介入规则;
- 能列出 AI 自动决定 / AI 推荐 / 必须人工各至少 3 项;
- 能画出 approval_packet 的目录结构。

4 从 TradingAgents 到 LQCD Research Group Agents

4.1 借鉴什么：TradingAgents 的核心设计思想

本项目借鉴了现代 multi-agent 系统（尤其是 TradingAgents）的设计理念。但借鉴的不是金融角色名称，而是背后的**系统架构思想**：

- **Specialized roles**: 不要一个大而全的 agent 做所有事。把分析任务拆成多个有明确边界的角色——执行者、诊断者、策略者、批判者、协调者。
- **Staged decision pipeline**: 数据检查 → 分析执行 → 诊断评估 → 批判审查 → 综合决策 → 审批 → 归档。每一步的输出是下一步的输入。
- **Debate / critic**: 系统必须设计反方角色，专门寻找漏洞和反例。不能让同一个 agent 既生成结果又自我审查。
- **Risk gate**: 所有 decision 都按风险等级分级处理。低风险自动通过，高风险必须人类审批。
- **Memory**: 保存结构化经验 (accepted / rejected / failed cases)，而不是保存聊天记录。为后续分析提供 prior experience。
- **Final decision gate**: 最终审批不是”每步都要人类点头”，而是在高风险节点保留人类最终责任。

4.2 角色映射表：从金融 Agent 到 LQCD Analysis Agents

下表说明 TradingAgents 的设计思想如何映射到本项目的 LQCD agent 体系。我们保留 TradingAgents 的设计理念，但使用适合 LQCD 科研组的新名称。

TradingAgents 设计	LQCD Agent 对应	迁移后的含义
Technical / fundamental / sentiment analysts	Correlator Analyst / Matrix Element Analyst / LQCD Physics Expert	从不同角度 (correlator 层面、bare ME 层面、整体 LQCD 物理) 分析同一物理问题
Bull researcher	Fit Strategy Postdoc (proponent 角色)	提出可行的分析方案和候选 fit hypotheses
Bear researcher	Critical Referee	专门寻找漏洞、反例和方法学风险
Trader	Analysis Coordinator / PI	综合多方信息形成行动建议，但不直接执行分析
Risk management team	LQCD Physics Expert + Critical Referee + Human Approval Gate	多层风险控制：物理合理性审查、方法学漏洞检查、人类最终审批

Portfolio manager

Human PI / advisor

最终批准高风险结论，承担最终
科学责任

重要说明：表格中保留了 TradingAgents 的原始角色名作为学术参考。本项目的正式 agent 名称均不使用金融术语（不出现 Bull、Bear、Trader、Portfolio Manager 等）。

4.3 为什么不用金融名字？

因为 LQCD 科研组不是一个 trading desk。学生需要理解的是**科研决策链**（诊断 → 策略 → 批判 → 协调 → 审批），而不是金融交易链（分析 → 看多/看空 → 交易 → 风控 → 组合管理）。本项目的 agent 命名原则是：**每个 agent 的名字和职责让人能立刻联想到真实 LQCD 合作组中的对应人员。**

本章交付清单：

- 能说出 TradingAgents 借鉴的 6 个核心设计思想；
- 能完成角色映射表（从金融角色到 LQCD agent）；
- 能解释为什么不用金融名字。

第三部分 Agent 角色设计

5 LQCD Research Group 风格的 Multi-Agent 架构

5.1 设计原则：像一个真实的 LQCD 分析子组

本项目的 agent 系统不是按照软件工程的角色（Developer、Reviewer、Manager）来组织，而是按照一个真实的 LQCD 分析子组（analysis subgroup）来组织。在一个典型的 LQCD 合作组中，分析一项 meson DA 任务通常涉及：会写代码和维护 pipeline 的研究生 / junior postdoc、负责检查 correlator 数据质量的分析者、负责设计 fit 策略的 postdoc、负责 bare matrix element 中间分析的分析者、对整体 LQCD 物理图像负责的 senior postdoc 或 professor、在需要时加入的微扰匹配 / 因子化专家、合作组内部的 internal referee、做最终判断的分析协调人或 PI、记录经验和教训的人。本项目的 agent 体系就是对这一真实科研组结构的映射。

5.2 Agent 体系总览（五层架构）

层级	Agent 名称		对应真实科研组中的角色
L1	Analysis Engineer Agent		会写代码和维护 pipeline 的研究生 / junior postdoc / coder
L2	Correlator Agent	Analyst	负责检查 correlator 数据质量的 postdoc / 高年级研究生
L2	Fit Strategy Agent	Postdoc	负责设计 fit 策略的 postdoc
L2	Matrix Element Analyst		负责 bare ME 中间分析的 postdoc / 高年级研究生
L3	Lattice QCD Expert Agent	Physics	senior postdoc / professor，负责整体 LQCD 物理判断
L3	Perturbative Matching Expert Agent		LaMET / 微扰 QCD / 因子化专家
L4	Critical Referee Agent		合作组 internal referee，专门挑错和寻找漏洞
L4	Analysis Coordinator / PI Agent		professor / project leader，综合决策
L4	Human Approval Gate		人类导师 / PI 的最终审批节点（risk-based conditional gate）
L5	Analysis Memory Curator		负责长期经验沉淀的人或系统

5.3 统一 Agent 介绍模板

以下每个 agent 使用统一模板介绍：

- **真实科研组类比：** 这个 agent 像组里的谁
- **解决的问题：** 为什么需要这个专门角色
- **输入 / 输出：** 结构化数据文件
- **可以自动做什么 / 不能做什么：** 职责边界
- **Month 1–2 最低实现：** rule-based 版本
- **未来高级实现：** LLM-based 或更高级版本
- **验收标准：** 如何判断这个 agent 工作正常

5.4 Agent 1: Analysis Engineer Agent (L1: 工程与执行)

真实科研组类比

会写代码和维护 pipeline 的研究生 / junior postdoc / coder。不是在干”低级 labor”，而是在建设科研基础设施。

解决的问题：科研分析不能长期依赖 notebook 里手动改路径、手动运行 cell、手动保存图。必须有可靠、可复现、可测试的工程基础。

输入：

- configs/<case_id>.yaml (Pydantic 验证后)
- raw data 路径 (只读)
- output directory 路径

输出：

- result.json
- summary.csv
- plots/ (effective mass、fit stability 等)
- run.log
- manifest.json

可以自动做：

- 读取 config、检查路径和文件是否存在
- 运行已有 notebook / script / CLI
- 把 notebook 重构成 Python CLI
- 生成结构化输出文件
- 修 bug、写 pytest
- 写 missing analysis scripts
- 维护 repo skeleton

不能做：

- 不决定 central fit
- 不决定是否接受物理结果
- 不修改 raw data
- 不覆盖 accepted results
- 不做大范围不可追踪 shell 操作

- 重大修改必须有 tests

Month 1 最低实现: config-driven CLI + Pydantic schema + pytest + 输出 manifest/result/log/plots。

未来高级实现: 自动检测缺失的 analysis scripts 并用 Codex 生成; 自动检测 pipeline 瓶颈; 支持 HPC 批量提交。

验收标准: 从 clean repo 运行一条命令复现 golden example; pytest 全部通过; 能解释 config 和 result 中每个字段; Git 有 8-12 个小 commit。

5.5 Agent 2: Correlator Analyst Agent (L2: 数据与拟合分析)

真实科研组类比

负责检查 correlator 数据质量的 postdoc / 高年级研究生。很多 LQCD 问题在 correlator 层面就已经能发现。

解决的问题: 在进入复杂拟合之前, 先判断数据质量是否足够。plateau 是否存在? bootstrap 误差是否合理? signal-to-noise 退化是否严重?

输入: 2pt correlator CSV / HDF5 文件 + config

输出:

- correlator_diagnostics.json
- effective_mass_summary.csv
- correlator_quality_report.md
- effective mass plots

可以自动做:

- 计算 effective mass 并画图
- 检查 plateau 是否存在 (基于数值判据)
- 检查 bootstrap / jackknife 误差是否合理
- 检查不同 momentum / z / gamma channel 的数据覆盖
- 检查 real / imaginary part 是否有明显异常
- 标记 bad correlator 或 suspicious correlator

不能做: 不做 fit (那是 Fit Strategy Postdoc 的工作); 不决定数据丢弃 (只能标记 suspicious)。

Month 1 最低实现: 基于现有 effective mass CLI, 加入 plateau 自动检测和 basic statistics reporting。输出 correlator_diagnostics.json。

未来高级实现: LLM-based correlator quality narrative; 对比多个 ensemble 的 correlator 质量; 自动检测异常 configuration。

验收标准: 能对 golden example 生成 correlator_diagnostics.json; 能标记至少一个可疑 correlator (如果存在); good case 和 bad case 的判断与导师一致。

5.6 Agent 3: Fit Strategy Postdoc Agent (L2: 数据与拟合分析)

真实科研组类比

负责设计 fit 策略的 postdoc。不是自动 grid search 工具，而是提出少量有物理动机的 fit hypotheses。

解决的问题：在给定 correlator diagnostics 后，如何选择 fit 方案？不是 brute-force scan 所有可能组合，而是提出 2–5 个有物理意义的候选方案。

输入：summary.csv + correlator_diagnostics.json

输出：

- fit_strategy_plan.json
- fit_candidates.csv
- fit_strategy_notes.md

输出示例：

```
{
  "recommended_fit_hypotheses": [
    {
      "name": "2state_default",
      "reason": "effective mass suggests excited-state contamination at small t",
      "tmin_range": [5, 7],
      "risk": "medium"
    },
    {
      "name": "1state_plateau_crosscheck",
      "reason": "late-time plateau appears stable but errors grow",
      "tmin_range": [8, 10],
      "risk": "medium"
    }
  ]
}
```

可以自动做：

- 选择 1-state / 2-state / 3-state fit 的候选方案
- 选择 $t_{\min}$ / $t_{\max}$ / ratio window
- 判断是否需要 prior
- 比较 plateau fit、two-state fit、summation fit
- 检查 fit 是否 overfit 或 underfit (基于 χ^2/dof 和参数灵敏度)
- 生成 2–5 个 high-value fit hypotheses

不能做：不搞 brute-force scan；不最终选定 central fit（那是 Analysis Coordinator 的工作）；不与 Critical Referee 争夺最终判断权。

Month 2 最低实现：rule-based module：读取 summary.csv，计算 tmin stability、model dependence sigma、chi2 quality。输出 fit_strategy_plan.json。

未来高级实现：LLM-based reasoning：从 effective mass 图和 correlator diagnostics 自动推理 fit 策略；学习先前 accepted cases 的 fit 选择模式。

验收标准：生成的 fit hypotheses 至少有 2 个；每个 hypothesis 都有物理理由；能在 3 个测试 case（good/ambiguous/bad）上输出合理的候选方案。

5.7 Agent 4: Matrix Element Analyst Agent (L2: 数据与拟合分析)

真实科研组类比

负责 bare matrix element 中间分析的 postdoc / 高年级研究生。meson DA 分析的关键不只在于 2pt fit，更在于从 ratio 走向物理 DA。

解决的问题：检查 bare matrix element 的 z -dependence、 P_z -dependence、real/imag symmetry 和 normalization。这些中间量的质量直接决定最终 DA 的可信度。

输入：ratio fit 结果 + bare ME output + fit_strategy_plan.json

输出：

- matrix_element_diagnostics.json
- bare_me_summary.csv
- z_dependence_plots.pdf
- pz_dependence_plots.pdf

可以自动做：

- 检查 ratio fit 结果是否在统计误差内稳定
- 检查 bare ME 的 z -dependence 是否平滑
- 检查 P_z -dependence 是否合理
- 检查 local matrix element normalization
- 检查 real / imaginary symmetry
- 检查不同 fit strategies 是否给出一致的 bare ME

不能做：不决定哪个 bare ME 是最佳结果；不做 matching 到 light-cone DA（那是 Perturbative Matching Expert 的工作）。

Month 2 最低实现：rule-based module：读取 bare ME，检查 z -dependence 平滑度和 real/imag symmetry。输出 matrix_element_diagnostics.json。

未来高级实现：LLM-based analysis：自动描述 z -dependence 趋势和异常；对比不同 P_z 下的 bare ME 一致性。

验收标准：能对 golden example 生成 matrix_element_diagnostics.json; 能标记 z -dependence 中的异常点（如果存在）；输出图能清楚展示 real/imag 两部分。

5.8 Agent 5: Critical Referee Agent (L4: 批判与决策)

真实科研组类比

合作组 internal referee。他的工作不是给好评，而是专门挑错。每个 analysis 在 promotion 之前都应该被 referee 严格审查。

解决的问题：科学研究中最危险的错误不是计算错误，而是没有发现假设的漏洞。必须有专门的批判角色，从方法学角度寻找所有潜在问题。

输入：fit_strategy_plan.json + matrix_element_diagnostics.json + summary.csv

输出：

- critical_referee_report.json
- objection_list.md

输出示例：

```
{
  "major_objections": [
    {
      "issue": "fit_window_selection",
      "claim": "The recommended tmin may be too early.",
      "evidence": "E0 shifts by 1.8 sigma between tmin=5 and tmin=7.",
      "requested_check": "Repeat with tmin>=7 and compare bare ME."
    }
  ],
  "minor_objections": [],
  "approval_recommendation": "revise_before_accept"
}
```

可以自动做（基于规则检查）：

- 检查 plateau 是否是假 plateau（例如只有 1-2 个点“看起来平”）
- 检查 fit window 是否 cherry-picking
- 检查 2-state fit 是否参数太多（overfitting）
- 检查 prior 是否过强（posterior $\approx$ prior）
- 检查 ratio window 是否避开了坏点
- 检查 P_z dependence 是否提示系统误差没控制
- 检查 z -dependence 是否有非物理振荡
- 检查 local matrix element 是否破坏 normalization

- 检查 conclusion 是否超过数据支持

不能做: 不决定最终方案 (只提 objection 和 requested check); 不替代 Analysis Coordinator 做综合判断。

Month 2 最低实现: rule-based module: 10+ 条 check rules。每条 rule 检查一个具体问题。输出 structured objection JSON。

未来高级实现: LLM-based reasoning: 生成自然语言的 referee report; 从 memory 中检索类似 case 的 objection 模式。

验收标准: 能在 good case 上给出 0 major objection; 能在 ambiguous case 上给出至少 1 个 major objection 并有具体 evidence; 能在 bad case 上正确给出”revise_before_accept”或”reject”。

5.9 Agent 6: Analysis Coordinator / PI Agent (L4: 批判与决策)

真实科研组类比

Professor / project leader。不做具体计算，但综合所有报告做决策：这个结果可以接受吗？需要重跑吗？需要升级到人类审批吗？

解决的问题: 多个 agent 产出了多种报告 (diagnostics、fit strategy、matrix element checks、referee objections)，需要有人综合所有信息做出决策。

输入: 所有下游 agent 的输出 + critical_referee_report.json

输出:

- decision_log.json
- coordinator_summary.md
- next_action_plan.json

decision_log.json 示例:

```
{
  "case_id": "l64c64a076_m140",
  "decision_type": "fit_recommendation",
  "candidate_fits": [...],
  "recommended_fit": "2state_tmin5_tmax15",
  "referee_flags": [...],
  "risk_level": "medium",
  "human_approval_required": true,
  "reasons": [
    "2-state fit is more stable than 1-state fit",
    "local matrix element passes sanity check",
    "Pz dependence needs human review"
  ],
  "targeted_rerun": {
    "needed": false,
```



```
"reason": null
}
}
```

可以自动做：

- 汇总所有 agent 报告
- 按规则判定 risk level
- 生成 decision_log
- 决定是否需要 targeted rerun
- 决定是否触发 Human Approval Gate
- 更新 case memory (accepted / rejected / pending)

不能做：

- 不亲自写代码
- 不忽略 Critical Referee 的 major objection
- 不把 medium/high risk 结果自动标为 accepted
- 不绕过 Human Approval Gate

Month 2 最低实现：rule-based module: 读取所有 agent 输出，计算 risk level，生成 decision_log 和 next_action_plan。

未来高级实现：LLM-assisted reasoning: 生成自然语言 summary；从 memory 中检索类似 case 的决策模式；自动生成 human-readable recommendation。

验收标准：能在 good case 上输出 low risk + 不触发 human approval；能在 ambiguous case 上输出 medium risk + human_review_recommended；能在 bad case 上输出 targeted rerun suggestion。

5.10 Agent 7: Human Approval Gate (L4: Risk-Based Conditional Gate)

本质

这不是一个“agent”，而是一个基于风险的、条件触发的审批机制。它是 exception path，不是 always-on sequential gate。

解决的问题：AI agents 可以处理大量 routine work 和越来越多的 analysis decisions，但最终的科学责任在人类。Human Approval Gate 确保高风险、不可逆、paper-level 的决定有人类把关。

触发条件：仅当 Analysis Coordinator 判定 risk level 为 High 或 Critical 时触发。Low/Medium risk 不触发。

系统端职责：

- 判断何时触发（基于 risk level）

- 生成完整 approval packet (7 个文件, 见 Ch.3)
- 在人类返回决定前将结果保存在 pending 状态
- 解析 07_human_decision.yaml 并更新 state / decision log / memory
- 超时未回复按 fallback 处理 (默认 hold_and_remind)

人类端职责:

- 审批 central fit 选择
- 审批 data rejection
- 审批 systematic uncertainty
- 审批 accepted result promotion
- 审批 paper-level conclusion
- 否决 AI 推荐时注明理由 (供 memory 学习)

Month 3 最低实现: 生成结构化 approval packet; 读取 human decision YAML; 在有/无人类决定的情况下正确更新 state。

未来高级实现: 交互式 Web UI; 邮件 / Slack 通知; decision deadline 自动提醒; 人类决策模式统计。

验收标准: high-risk case 自动生成 approval packet; 无 human decision 时 state 正确停留在 pending; 人类决策被正确解析并推进 state。

5.11 Agent 8: Analysis Memory Curator (L5: 长期学习)

真实科研组类比

负责记录经验教训的组内成员。不产出当次分析的结果, 但让下一次分析更高效。

解决的问题: 没有 memory 的系统每次分析都从零开始。Memory Curator 把每次分析的 accepted/rejected/failed cases 结构化保存, 让系统逐步积累经验。

输入: decision_log + human feedback + agent 输出路径

输出:

- memory/accepted_cases.yaml
- memory/rejected_cases.yaml
- memory/pending_cases.yaml
- memory/failure_patterns.md
- memory/memory_update_log.json

可以自动做:

- 保存 accepted / rejected / failed cases

- 总结失败原因（基于 Critical Referee flags）
- 把 human feedback 写入对应 case
- 检索类似 case 供 Analysis Coordinator 参考

不能做：不覆盖人类决定；不删除历史记录。

Month 3 最低实现：YAML / JSON 文件存储；basic 检索（按 ensemble、momentum、risk level）。

未来高级实现：SQLite / DuckDB 存储；RAG 检索；自动生成 failure pattern summary；为相似 case 提供 prior fit strategy recommendation。

验收标准：每个 case 结束后 memory 自动更新；能通过命令行检索过往 cases；accepted/rejected/pending 三类 cases 分开保存。

5.12 Agent 9 & 10: Lattice QCD Physics Expert + Perturbative Matching Expert (L3: 高级物理审查)

定位

这两个 agent 在前三个月做简化版（basic rule-based checks），后续升级为重点开发方向。

Lattice QCD Physics Expert Agent（对应 professor / senior postdoc）：

- 判断 ensemble / lattice spacing / volume / pion mass 是否适合当前分析
- 判断 discretization effect、finite-volume effect、excited-state contamination 风险
- 检查结果是否与 LQCD 基本物理直觉冲突
- 输出：lqcd_physics_review.json + systematics_risk_report.md
- 前三月：basic rule-based checks（例如检查 E_0 是否在合理范围）
- 未来：LLM-based LQCD physics reasoning

Perturbative Matching Expert Agent（对应 LaMET / pQCD / factorization expert）：

- 检查 quasi-DA 到 light-cone DA matching setup
- 检查 renormalization scheme 和 matching scale
- 检查 perturbative kernel 使用是否正确
- 输出：matching_review.json + matching_systematics_report.md
- 前三月：放在 advanced roadmap 中，暂不实现
- 未来：quasi-DA $\rightarrow$ physical light-cone DA 全流程自动化审查

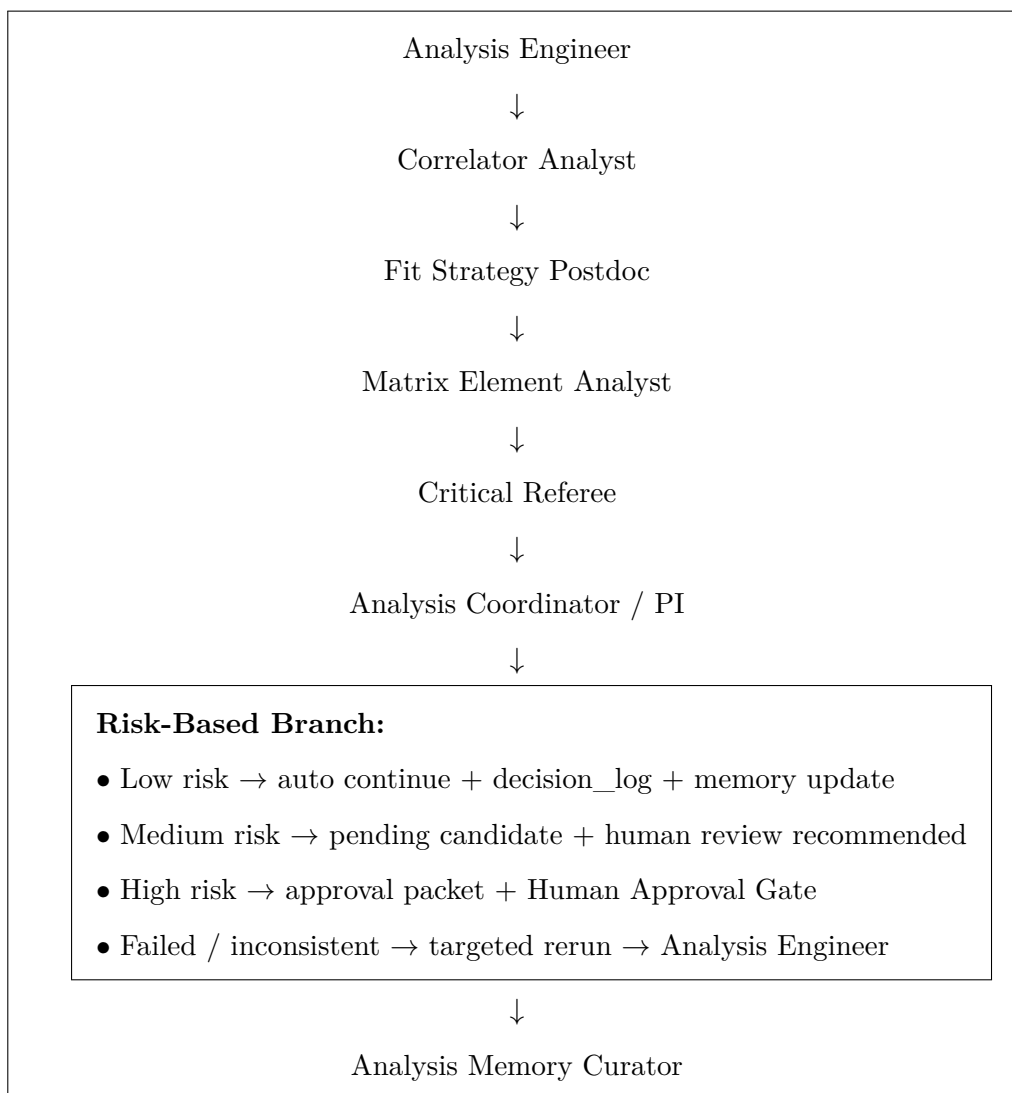
本章交付清单：

- 能说出 10 个 agent 的名字、层级和对应的真实科研角色；
- 每个 agent 能说出至少 3 项“可以自动做”和 2 项“不能做”；

- 能画出 agent 之间的输入输出依赖关系。

6 Agent workflows: 从数据到决策

6.1 第一阶段：真实数据分析 pipeline (Month 1–2 实现)



6.2 第二阶段：加入理论 matching (Month 3+)

在 Matrix Element Analyst 和 Critical Referee 之间插入：

Matrix Element Analyst → **Lattice QCD Physics Expert** → **Perturbative Matching Expert** → Critical Referee → ...

6.3 第三阶段：多 Agent Debate (Advanced roadmap)

Fit Strategy Postdoc A (proponent): “2-state fit should be central. Ground state is stable and χ^2/dof is acceptable.”

Fit Strategy Postdoc B (alternative): “Late-time 1-state fit should be used as central. 2-state introduces extra parameters with limited gain in the fit window.”

Critical Referee: “Both are not sufficiently justified because P_z dependence changes

after normalization. Neither proposal addresses this.”

Analysis Coordinator / PI (综合后判定 high risk): “Request targeted rerun with $t_{\min} \geq 7$ and compare bare ME before making a final decision. Generate approval packet for human review.”

Human PI: “Approve 2-state as central with a Pz-dependence systematic study added to next round. Record this debate in memory.”

本章交付清单:

- 能画出三个阶段的数据流图;
- 能解释 Analysis Coordinator 后的条件分支逻辑;
- 能给出一个 multi-agent debate 的场景示例。

第四部分 技术栈地图

7 工具、职责与选型理由

本章按**功能**组织工具介绍，而不是按工具名堆砌。每个工具回答三个问题：它解决什么问题？本项目什么时候用？学生最低需要掌握什么？

7.1 Reproducible Analysis: config、schema 与结构化输出

解决的问题：科研分析不能依赖 notebook 里的手动路径和手动保存。可复现分析的基石是 config-driven pipeline + typed schema + structured output。

核心工具：

- **Python CLI:** 所有分析通过命令行启动, 参数来自 config 文件。例: `python -m lqcd_agent.analysis.ru --config configs/l64c64a076_m140.yaml`
- **Pydantic:** 定义 config schema 和 result schema。所有输入输出都是 typed、validated、self-documenting。
- **YAML / JSON / CSV:** config 用 YAML (人类可读), result 用 JSON (机器可读), summary 用 CSV (表格分析)。

使用时序: Month 1 Week 2 (CLI + schema)、贯穿全部三个月。

学生最低需要掌握: 能写一个 Pydantic BaseModel; 能用 argparse 或 click 写 CLI entry point; 能解释为什么 config 必须包含所有影响结果的参数。

不要求一开始掌握: 复杂的 Pydantic validator; nested generic types; async CLI。

7.2 Testing: pytest

解决的问题: Codex 生成的代码不能直接信任。pytest 把关键行为固定为可回归的测试, 防止后续修改无声破坏已有功能。

核心工具: pytest + tmp_path (临时目录测试)

使用时序: Month 1 Week 2 开始写 config validation test; Week 3 加入 CLI smoke test 和 output existence test; Month 2-3 持续扩展。

学生最低需要掌握: 能写 smoke test、unit test、regression test; 能用 tmp_path 避免测试污染真实输出。

不要求一开始掌握: fixture factories; parametrize; mocking; coverage reports。

7.3 Scientific Workflow: Snakemake

解决的问题: 分析有多个步骤, 步骤之间有文件依赖。改了一个参数后, 不知道哪些步骤需要重跑。Snakemake 管理文件依赖 DAG, 只重跑需要更新的步骤。

为什么选 Snakemake 而不是 Dagster/Prefect:

- Snakemake 专为 scientific file-driven workflow 设计 (raw data → result → plot → summary)

- 用 Python 风格规则，适合 LQCD 学生逐步学习
- 原生支持 HPC 环境 (Slurm、Condor 等)
- Dagster/Prefect 更偏数据工程平台，对文件型科研流程过重

使用时序：Month 2 Week 5 引入。

学生最低需要掌握：能写 rule (input/output/shell)；能理解 dependency-aware rerun；能用 `--dry-run` 检查计划；能把已有 CLI 包装成 Snakemake rule。

不要求一开始掌握：checkpoint files；wildcards；复杂的 HPC profile；conda integration。

7.4 Agent Orchestration: LangGraph (或 Python State Machine)

解决的问题：多个 agent 之间如何传递状态？如何在正确节点暂停等待人类审批？如何在 rerun 后恢复状态？LangGraph (或等价 state-machine) 管理 agent 决策 DAG。

为什么选 LangGraph：

- 适合 stateful multi-agent workflow
- 支持 durable execution (状态持久化)
- 支持 human interrupt / resume
- agent 流程表示为清晰 graph，不是不可追踪的长聊天

与 Snakemake 的分工：

工具	负责什么	不负责什么
Snakemake	科学计算 workflow、文件依赖、增量重跑、HPC 批量执行	不负责 agent reasoning 和 human approval 对话
LangGraph	多 agent 状态流、判断节点、人工中断、targeted rerun、memory update	不负责底层文件依赖和大规模批处理

即：Snakemake 管数据分析 DAG，LangGraph 管 agent 决策 DAG。

使用时序：Month 3 Week 9 引入。Month 2 可以先用简单的 Python state machine。

学生最低需要掌握：能定义 typed state；能写 node 函数；能用 conditional edge；能保存/恢复 graph state。

不要求一开始掌握：LangGraph 全部高级 API；streaming；checkpoint 高级配置。

7.5 Memory: 从 YAML 到 SQLite/DuckDB/RAG

解决的问题：没有 memory，每次分析都从零开始。memory 保存结构化经验（不只是聊天记录）。

演进路线：

- **Month 1–2：**不需要独立的 memory 模块。output 目录 + decision_log.json 即为初始 memory。

- **Month 3:** YAML / JSON 文件存储 (`accepted_cases.yaml` 等)。
- **未来:** SQLite / DuckDB (支持检索和统计) + RAG (让 agent 读取历史 decisions)。

学生最低需要掌握: 能定义 memory entry schema; 能写入/读取 YAML; 能按 `case_id` 检索。

7.6 Observability: `run.log`、`decision_log.json` 与未来 tracing

解决的问题: 系统运行后, 如何回答”上一步发生了什么? 为什么做了这个决定? 哪里最常失败?”

当前方案:

- `run.log`: 命令、返回码、运行时间、错误
- `manifest.json`: config hash、git commit、timestamp、输出文件列表
- `decision_log.json`: 每次 decision 的理由、证据、风险等级

未来方案: MLflow 或 OpenTelemetry 风格 tracing, 记录每个 node 耗时、LLM 调用成本、失败率分布。

7.7 AI-Assisted Development: Codex

解决的问题: 学生不应该手工写每一行代码。Codex 辅助生成 repo skeleton、CLI、schema、tests、Snakemake rules、agent nodes。

使用时序: 贯穿全部三个月。每天一个小任务, Codex 生成初版 → 学生检查 → 运行 pytest → Git commit。

学生最低需要掌握: 能写结构化 prompt (见 Part VI Ch.11); 能用 traceback 让 Codex 修 bug; 能检查 Codex 代码的正确性和安全性。

Codex 不应该直接决定: central fit、数据丢弃、systematic uncertainty、accepted result promotion、paper-level physical conclusion、未经测试的大范围重构。

7.8 技术栈全景图

功能层	工具	引入时间	学生最低要求
Analysis Core	Python CLI + Pydantic	Month 1 W2	config-driven pipeline
Testing	pytest	Month 1 W3	smoke + unit test
Workflow	Snakemake	Month 2 W5	rule + dependency
Agent State	LangGraph / state-machine	Month 3 W9	typed state + conditional edge
Memory	YAML / JSON	Month 3 W10	structured cases
Observability	run.log + decision_log	Month 1+	provenance recording
AI Dev	Codex	Throughout	structured prompting

本章交付清单：

- 能解释 Snakemake 和 LangGraph 的分工（一个管计算 DAG，一个管决策 DAG）；
- 能说出每个工具在本项目中什么时候引入；
- 能画出技术栈的分层结构。

第五部分 三个月实战课程

本章是三个月实战课程的核心。每个 Month 包含目标、核心 agent、每周安排、最终交付物、验收标准、每周的具体任务与命令、常见错误。

如何使用本部分：

- 每个 Week 开始前，先读本节了解本周目标和任务；
- 然后参考对应的详细 month guide (lqcd_agent_monthN_detailed_guide_CN.pdf) 中的逐日任务单和物理背景；
- 每天按 Codex prompt 模板让 AI 生成代码，参考 Part VI 的练习设计；
- 每周结束时用验收标准自查，向导师展示当周产出。

8 Month 1: 从 Golden Example 到可复现 CLI

8.1 Month 1 目标

第一个月的目标不是从零发明新 physics，而是把一个已经准备好的真实分析流程变成学生能复现、能测试、能解释、能扩展的工程化系统。

核心 agent 在本月实现：

- **Analysis Engineer Agent**: CLI、config、schema、pytest、structured output
- **Correlator Analyst Agent**: effective mass diagnostics、correlator quality report

Month 1 完成后系统能力：

1. 从 clean repo 运行一条命令复现 golden example
2. 自动生成 result.json、summary.csv、plots、run.log
3. 自动生成 correlator_diagnostics.json 和 effective mass plots
4. pytest 保护核心功能不受破坏

8.2 Week 1: 环境、Golden Example 复现与 Data-Flow Map

学习目标：

- 理解 repo 结构和 golden example 的分析链
- 能手动复现 golden example 并记录每一条命令
- 能画出 data-flow map

具体任务：

1. **Day 1: 环境准备与 repo 初读。** clone repo、创建 Python 环境、安装依赖、运行现有测试、阅读 README。产出 docs/repo_initial_notes.md。
2. **Day 2: 手动复现 golden example。** 按现有方式运行 examples/l64c64a076_m140/ 的每一步，记录命令、输入文件、输出文件。产出 docs/golden_example_reproduction.md 和 run.log。
3. **Day 3: 画 data-flow map。** 把分析流程拆成输入 → 处理 → 输出。产出 docs/data_flow_map.md。
4. **Day 4: 识别 config 参数。** 检查现有脚本中哪些值应该变成 config。产出 docs/config_design_notes.md 和 draft YAML。
5. **Day 5: Week 1 总结。** 整理第一周报告。产出 docs/week1_report.md。

命令示例：

```
git status
python --version
pytest
python -m lqcd_analysis.cli 2pt-effective-mass ...
```

输出文件：

```
docs/repo_initial_notes.md
docs/golden_example_reproduction.md
docs/data_flow_map.md
docs/config_design_notes.md
docs/week1_report.md
```

导师检查问题：

- 学生是否真的跑通了一遍 golden example?
- 是否记录了完整命令和 git commit?
- 是否理解了主要输入输出?
- 是否知道哪些参数应该 config 化?

常见错误：

- 只跑 notebook 不记录命令 → 无法复现
- 一开始就让 Codex 写完整系统 → 代码过大、不可控

8.3 Week 2: Config-Driven CLI + Pydantic Schema**学习目标：**

- 能把 golden example 的入口从”手动运行 cell”变成”config-driven CLI”
- 能定义 Pydantic config schema 和 result schema

具体任务：

1. **定义 config schema**。创建 `src/lqcd_agent/schema/analysis_config.py`。Pydantic BaseModel，验证所有字段。写 `test_schema.py`。
2. **定义 result schema**。创建 `src/lqcd_agent/schema/analysis_result.py`。包含 `run_id`、`status`、`git_commit`、`outputs`、`warnings`、`errors`。
3. **实现最小 CLI**。创建 `src/lqcd_agent/cli/run_golden.py`。命令：`python -m lqcd_agent.cli.run_golden --config configs/golden_example.yaml`。
4. **加入 manifest 和 log**。每次运行写 `manifest.json` (timestamp, git commit, config path, environment) 和 `run.log`。
5. **记录 Codex 使用日志**。维护 `docs/codex_usage_log.md`。

命令示例：

```
python -m lqcd_agent.cli.run_golden --config configs/l64c64a076_m140.yaml
pytest tests/test_schema.py -q
```

输出文件：

```
src/lqcd_agent/schema/analysis_config.py
src/lqcd_agent/schema/analysis_result.py
src/lqcd_agent/cli/run_golden.py
configs/l64c64a076_m140.yaml
tests/test_schema.py
docs/codex_usage_log.md
```

导师检查问题：

- config 是否完整？所有影响结果的参数是否都进入了 config？
- Pydantic validation 是否会对错误 config 清晰报错？
- CLI 是否能从 clean environment 运行？
- 是否有 overwrite protection？

常见错误：

- config 字段太随意 → 用 Pydantic Field 加约束
- 没有 schema 直接读 dict → 后续 agent 无法可靠读取

8.4 Week 3: Correlator Diagnostics + pytest**学习目标：**

- 能实现 Correlator Analyst 的 rule-based 版本
- 能用 pytest 保护关键行为

具体任务：

1. **实现数据完整性检查。**创建 src/lqcd_agent/audit/data_audit.py。检查 input root、必要文件、output dir 安全性。输出 audit_report.json。
2. **实现 effective mass diagnostics。**基于现有 effective mass CLI，加入 plateau 自动检测。输出 correlator_diagnostics.json 和 effective_mass_summary.csv。
3. **扩展 pytest。**至少 5 个有效测试：valid config、invalid config、missing input、CLI smoke test、output existence。
4. **定义输出规范。**规定每次运行必须输出：manifest.json、result.json、summary.csv、run.log、audit_report.json、plots/。

命令示例：

```
pytest tests/ -q
python -m lqcd_agent.audit.run --config configs/l64c64a076_m140.yaml
python -m lqcd_agent.analysis.correlator_diagnostics \
    --config configs/l64c64a076_m140.yaml
```

输出文件：

```
src/lqcd_agent/audit/data_audit.py
src/lqcd_agent/analysis/correlator_diagnostics.py
tests/test_data_audit.py
tests/test_cli_smoke.py
outputs/<case_id>/correlator_diagnostics.json
outputs/<case_id>/effective_mass_summary.csv
```

导师检查问题：

- data audit 是否能检测缺失输入？
- correlator diagnostics 是否能标记 bad correlator？
- pytest 是否覆盖正常和失败情况？
- 学生能否解释为什么 raw data 不能被修改？

常见错误：

- 用真实数据路径写测试 → 测试不可迁移；用 tmp_path
- 把 Analysis Engineer 当成物理判断者 → 它只执行和检查，不做 fit 选择

8.5 Week 4: Month 1 Integration Demo

学习目标：

- 能用一个命令跑通 Month 1 全流程
- 能写 Month 1 final report

具体任务：

1. **集成 Analysis Engineer wrapper**。创建 src/lqcd_agent/analysis_engineer/。AnalysisEngineerTask → AnalysisEngineerTaskResult。
2. **写 Month 1 final report**。内容：项目目标理解、golden example 复现结果、data-flow map、config schema 设计、result schema 设计、data audit、pytest 列表、Analysis Engineer wrapper 雏形。
3. **准备 demo**。10–15 分钟展示：golden example 流程、repo 结构、CLI 命令、result.json/summary.csv/plots、pytest 结果。

命令示例：

```
python -m lqcd_agent.analysis_engineer.run \
    --task configs/tasks/month1_golden_task.yaml
pytest -q
```

Month 1 最终交付物：

```
configs/l64c64a076_m140.yaml      # golden example 配置
src/lqcd_agent/schema/             # Pydantic schema
```

```
src/lqcd_agent/cli/run_golden.py    # CLI 入口
src/lqcd_agent/audit/data_audit.py  # 数据检查
src/lqcd_agent/analysis_engineer/   # Analysis Engineer wrapper
tests/                               # 5+ pytest tests
outputs/<case_id>/
  manifest.json
  result.json
  summary.csv
  correlator_diagnostics.json
  effective_mass_summary.csv
  run.log
  plots/
docs/month1_report.md
```

Month 1 验收标准:

- 从 clean repo 运行一条命令复现 golden example
- 解释 config 中每个字段
- 解释 result.json 中每个字段
- 运行 pytest 并说明每个测试保护什么
- 展示至少 8-12 个小 commit
- 说明 Codex 生成了什么、自己检查了什么、改了什么

9 Month 2: Workflow、Postdoc-level Diagnostics 与 Decision Log

9.1 Month 2 目标

第二个月把 Month 1 的 CLI 升级为可复现 workflow, 并实现 Fit Strategy Postdoc、Matrix Element Analyst、Critical Referee 和 Analysis Coordinator 的 rule-based 版本。

核心 agent 在本月实现:

- **Fit Strategy Postdoc Agent:** fit strategy plan + fit candidates
- **Matrix Element Analyst Agent:** bare ME diagnostics + z/Pz-dependence checks
- **Critical Referee Agent:** objection list + referee report
- **Analysis Coordinator / PI Agent:** decision log + risk level + targeted rerun

Month 2 完成后系统能力:

1. Snakemake workflow 管理全部分析步骤的依赖
2. Fit Strategy Postdoc 生成 fit_strategy_plan.json
3. Matrix Element Analyst 生成 matrix_element_diagnostics.json
4. Critical Referee 生成 critical_referee_report.json
5. Analysis Coordinator 生成 decision_log.json
6. 支持 targeted rerun suggestion

9.2 Week 5: Snakemake Workflow

学习目标: 能把 Month 1 CLI 包装成 Snakemake workflow, 支持依赖管理和增量重跑。

任务: 整理 Month 1 命令流 → 创建 Snakefile → 拆分 rules → dry-run 验证 → 真实运行。

命令示例:

```
snakemake -s workflow/Snakefile \
  --configfile configs/l64c64a076_m140.yaml --cores 1 --dry-run
snakemake -s workflow/Snakefile \
  --configfile configs/l64c64a076_m140.yaml --cores 1 --rerun-incomplete
```

输出: workflow/Snakefile、workflow/rules/*.smk

导师检查: dry-run 是否展示正确依赖图? rule 中是否有复杂分析逻辑 (应只调用 CLI)?

常见错误: Snakefile 里写太多 Python 分析逻辑 → rule 只负责调度, 分析逻辑保留在 src/lqcd_agent/ 中。

9.3 Week 6: Fit Strategy Postdoc Agent

学习目标: 实现 rule-based Fit Strategy Postdoc, 从 summary.csv 生成 fit_strategy_plan.json。

任务：统一 summary.csv 字段 → 实现 fit diagnostics metrics (χ^2 quality、tmin stability、model dependence sigma) → 生成 fit hypotheses。

命令示例：

```
python -m lqcd_agent.analysis.fit_strategy_postdoc \
--summary outputs/l64c64a076_m140/summary.csv \
--output outputs/l64c64a076_m140/fit_strategy_plan.json
```

输出：fit_strategy_plan.json、fit_candidates.csv、fit_strategy_notes.md

导师检查：生成的 hypotheses 是否有物理理由？是否避免了 brute-force scan？

常见错误：把 Fit Strategy Postdoc 变成 grid search 工具 → 它应该提出 2-5 个有物理动机的候选，而不是扫描全部组合。

9.4 Week 7: Matrix Element Analyst + Critical Referee

学习目标：实现 bare ME diagnostics 和 referee checks。

任务：实现 Matrix Element Analyst (z-dependence、Pz-dependence、real/imag symmetry) → 实现 Critical Referee (10+ rules) → 构造 3 个测试 case。

命令示例：

```
python -m lqcd_agent.analysis.matrix_element_analyst \
--bare-me outputs/l64c64a076_m140/bare_me.csv \
--output outputs/l64c64a076_m140/matrix_element_diagnostics.json
python -m lqcd_agent.critical_referee.check \
--diagnostics outputs/l64c64a076_m140/ \
--output outputs/l64c64a076_m140/critical_referee_report.json
```

输出：matrix_element_diagnostics.json、bare_me_summary.csv、critical_referee_report.json、objection_list.md

导师检查：Critical Referee 是否真的在找问题（不只是重复 Fit Strategy Postdoc 的结论）？3 个测试 case (good/ambiguous/bad) 是否覆盖？

常见错误：Critical Referee 只重复 Fit Strategy Postdoc 的结论 → Critical Referee 的任务是找风险和漏洞，不是复述已有结论。

9.5 Week 8: Analysis Coordinator + Decision Log + Month 2 Demo

学习目标：实现 Analysis Coordinator 的 rule-based 版本，综合所有 agent 报告输出 decision_log。

任务：定义 decision schema → 实现 risk level 判定规则 → 生成 decision_log 和 coordinator_summary.md → 集成到 Snakemake workflow → Month 2 demo。

判定规则示例：

- Analysis Engineer failed → risk=high, 建议 rerun/debug
- 所有 fit failed → risk=high, human approval required

- χ^2/dof 普遍过大 $\rightarrow$ risk=high
- model dependence 超 2 sigma $\rightarrow$ risk=medium or high
- 2-state 稳定且 1-state 不稳定 $\rightarrow$ provisional 推荐 2-state, medium risk
- Critical Referee 提出 high severity flag $\rightarrow$ human approval required

命令示例:

```
python -m lqcd_agent.analysis_coordinator.run \
  --case outputs/l64c64a076_m140/case_manifest.json
```

Month 2 最终交付物:

```
workflow/Snakefile + rules/*.smk
src/lqcd_agent/analysis/fit_strategy_postdoc.py
src/lqcd_agent/analysis/matrix_element_analyst.py
src/lqcd_agent/critical_referee/critical_referee.py
src/lqcd_agent/analysis_coordinator/
outputs/<case_id>/
  fit_strategy_plan.json
  fit_candidates.csv
  matrix_element_diagnostics.json
  critical_referee_report.json
  decision_log.json
  coordinator_summary.md
  next_action_plan.json
tests/test_fit_strategy_postdoc.py
tests/test_critical_referee.py
tests/test_analysis_coordinator.py
memory/accepted_cases.yaml
memory/rejected_cases.yaml
docs/month2_report.md
```

Month 2 验收标准:

- `snakemake --cores 1` 复现 golden example
- 修改 config 后只重跑必要下游步骤
- Analysis Coordinator 输出 `decision_log` (含理由和 evidence)
- Critical Referee 能指出至少三类潜在问题
- 至少有 good/ambiguous/failed 三种测试 case
- 学生能解释 Snakemake 和 LangGraph 的区别

10 Month 3: Multi-Agent Product Demo

10.1 Month 3 目标

第三个月把前两个月的模块组织成 stateful multi-agent prototype, 支持 Human Approval Gate、case memory、targeted rerun、evaluation harness 和 final demo report。

核心内容在本月实现:

- Human Approval Gate (approval packet 生成 + 人类决策解析)
- Analysis Memory Curator (YAML/JSON memory)
- LangGraph 或 Python state-machine (agent 状态流编排)
- Evaluation harness (good/ambiguous/bad cases)
- Final demo report

Month 3 完成后系统能力:

1. LangGraph (或 Python state-machine) 编排完整的 agent 决策流
2. Low risk 自动推进; Medium risk 标记 pending; High risk 生成 approval packet
3. Case memory 保存 accepted/rejected/pending cases
4. Evaluation harness 用固定 cases 测试 agent 决策稳定性
5. 一键 demo 脚本展示完整流程

10.2 Week 9: Python State Machine / LangGraph Prototype

学习目标: 能把 Month 2 的模块封装成 state machine nodes, 实现结构化状态传递。

任务: 定义 AgentState (TypedDict) → 封装已有模块为 node 函数 → 实现线性 graph → 保存 graph_state.json。

命令示例:

```
python -m lqcd_agent.graph.run_graph \
  --case configs/cases/164c64a076_demo.yaml \
  --output-dir outputs/graph_demo/164c64a076
```

AgentState 关键字段:

```
class AnalysisAgentState(TypedDict):
    case_id: str
    status: str # "running" / "success" / "failed" / "
needs_human_approval"
    risk_level: str # "low" / "medium" / "high" / "critical" / "
unknown"
    human_approval_required: bool
    correlator_diagnostics_path: str | None
```

```

fit_strategy_path: str | None
matrix_element_diagnostics_path: str | None
critical_referee_report_path: str | None
decision_log_path: str | None
rerun_requested: bool
errors: list[dict]

```

输出: outputs/.../graph_state.json、graph_run.log

导师检查: 每个 node 的输入输出是否结构化 (不是自由文本)? state 是否能被保存和恢复?

常见错误: node 之间传递随意自由文本 → 必须用 typed state 字段。

10.3 Week 10: Human Approval Gate + Case Memory

学习目标: 实现 risk-based conditional gate: Analysis Coordinator 后根据 risk level 走不同分支。

任务: 实现条件分支逻辑 → 生成 approval packet (7 文件) → 解析 human decision YAML → 更新 case memory。

条件分支逻辑:

```

if risk_level == "low":
    auto_continue()          # 自动推进
    write_decision_log()
    memory_update("accepted")
elif risk_level == "medium":
    continue_as_pending()    # 标记 pending
    write_decision_log()
    memory_update("pending")
elif risk_level == "high":
    generate_approval_packet()
    await_human_decision()   # 暂停, 等待审批
elif risk_level == "critical":
    halt_and_notify_pi()     # 禁止推进

```

输出: approval_packet/(7 个文件)、memory/accepted_cases.yaml、memory/rejected_cases.yaml、memory/pending_cases.yaml

导师检查: high risk case 是否生成 approval packet? 无 human decision 时 state 是否停留在 "needs_human_approval"? medium risk 结果是否不自动进入 accepted?

常见错误: 把 Human Approval Gate 当成 always-on gate → 它只在 high/critical risk 触发。

10.4 Week 11: Evaluation Harness + Final Report 自动生成

学习目标: 能用固定的 evaluation cases 测试 agent 决策稳定性, 能自动生成 human-readable report。

任务：构造 3 类固定 cases (good/ambiguous/bad) → 实现 run_evaluation.py → 实现 generate_report.py → 运行 evaluation 并生成报告。

Evaluation cases:

- **good_plateau:** plateau 稳定、fit 质量好 → expected risk = low、human approval = false
- **ambiguous_excited_state:** 1-state/2-state 有差异 → expected risk = medium/high、human approval = true
- **bad_no_plateau:** 无稳定 plateau、所有 fit 失败 → expected risk = high、decision 包含 "rerun" 或 "reject"

命令示例：

```
python evaluation/run_evaluation.py \
--cases evaluation/cases \
--output-dir outputs/evaluation
```

输出：evaluation_summary.csv、evaluation_report.md

导师检查：Analysis Coordinator 行为是否与 expected behavior 一致？mismatch 是否被清楚报告？

10.5 Week 12: Final Demo + Documentation + Productization

学习目标：整理一键 demo、README、architecture diagram、future roadmap。

任务：写一键 demo 脚本 → 补全 README → 画 architecture diagram → 准备最终汇报。

一键 demo 命令：

```
bash scripts/run_month3_demo.sh
```

Final demo 必须展示：

1. 一个 golden example case 从输入到 decision 的完整过程
2. 一整条 pipeline 运行 (CLI + Snakemake + agent graph)
3. multi-agent decision flow (每个 node 的输入输出)
4. 一份 decision_log (含理由、risk level、human approval flag)
5. 一个 critical objection (Critical Referee 的 major objection)
6. 一次 targeted rerun suggestion
7. 一个 human approval packet
8. 一个 case memory update
9. 一份 final human-readable report

Month 3 最终交付物：

```
src/lqcd_agent/graph/          # state machine / LangGraph
src/lqcd_agent/memory/        # case memory
outputs/<case_id>/
  graph_state.json
  approval_packet/            # 7 个文件
  final_report.md
memory/
  accepted_cases.yaml
  rejected_cases.yaml
  pending_cases.yaml
evaluation/
  cases/                      # good/ambiguous/bad
  expected/
  run_evaluation.py
  evaluation_report.md
scripts/run_month3_demo.sh
docs/architecture.md
docs/month3_report.md
```

Month 3 验收标准:

- 一个完整 agent run (从 config 到 decision)
- graph 中每个 node 有明确的输入输出
- 一次 high-risk case 触发 Human Approval Gate (含 approval packet 生成)
- 一次 Critical Referee 触发 targeted rerun
- decision log 和 memory update 正确记录
- evaluation harness 覆盖 good/ambiguous/bad 三类 cases
- 学生能清晰解释系统的边界、风险和未来升级方向

第六部分 Codex 实战练习

11 10 个 Codex Prompt 练习

本章提供 10 个结构化 Codex 练习。每个练习包含目标、输入材料、Codex prompt、禁止事项、验收标准。这些练习穿插在 Part V 的周任务中使用。

11.1 练习 1: 生成 Pydantic Config Schema

目标: 让 Codex 从 draft YAML 生成 Pydantic schema。

输入: configs/golden_example_draft.yaml

Codex prompt:

```
We are building an AI-first LQCD meson DA analysis-agent system.
Current stage: Month 1 Week 2.

Task:
Implement Pydantic config schemas in `src/lqcd_agent/schema/analysis_config.py`.

Use the draft YAML config in `configs/golden_example_draft.yaml` as reference.

Requirements:
- Define nested schemas for ensemble metadata, IO paths, analysis selections,
  fit settings, and runtime options
- Validate that output_dir is not empty
- Validate that bootstrap_samples is positive
- Validate that z_values are integers
- Add a helper function load_config(path: str) -> GoldenExampleConfig
- Add pytest tests for valid and invalid config

Do not change the analysis algorithm yet.
Do not modify raw data paths.
```

禁止事项: 不允许修改已有分析代码；不允许引入非必要依赖。

验收标准: YAML config 可以被 load；错误 config 会清楚报错；pytest 覆盖基本 validation。

11.2 练习 2: 写 Python CLI

目标: 让 Codex 生成 CLI entry point。

输入: 已有的分析函数 + config schema

Codex prompt:

```
Task:
Implement a minimal CLI for running the golden example:
`src/lqcd_agent/cli/run_golden.py`
```


Requirements:

- Command: ``python -m lqcd_agent.cli.run_golden --config configs/golden_example.yaml``
- Load and validate the Pydantic config
- Create output directory
- Refuse to overwrite existing output unless `allow_overwrite` is true
- Write `manifest.json` (`run_id`, `timestamp`, `git commit`, `config path`, `environment`)
- Write `run.log`
- Call placeholder or existing script hook for analysis
- Write `result.json` using the result schema
- Add CLI smoke test using `tmp_path`

禁止事项: 不允许修改 raw data; 不允许实现 Snakemake。

验收标准: CLI 能从 config 启动; `manifest/result/log` 正确生成; smoke test 通过。

11.3 练习 3: 写 pytest

目标: 让 Codex 为已有模块生成测试。

输入: 已有模块的代码和 docstring

Codex prompt:

Task:

Add pytest tests for the existing data audit module.

Focus on:

- valid input (all files present)
- missing input_root
- missing required file
- output directory creation
- `audit_report.json` content validation
- CLI smoke test using `tmp_path`

Do not change the implementation unless a test reveals a real bug.

If you find a bug, explain it first and then make a minimal fix.

禁止事项: 不允许修改已有函数签名; 不允许依赖真实数据路径。

验收标准: pytest 全部通过; 测试覆盖正常和失败路径; 测试可在任何机器上运行。

11.4 练习 4: 用 Traceback 修 Bug

目标: 训练学生用 traceback 驱动 Codex 修 bug。

输入: 失败命令 + 完整 traceback + 期望行为

Codex prompt:

The following command failed:

```
python -m lqcd_agent.cli.run_golden --config configs/golden_example.yaml
```

Traceback:

[paste full traceback]

Expected behavior:

The CLI should load the config, run the golden example, and write manifest.json.

Please fix only the cause of this failure.

Do not refactor unrelated code.

If appropriate, add a regression test so this failure is caught in the future.

禁止事项：不允许大面积重构；不允许修改 traceback 中未涉及的文件。

验收标准：同样命令不再报错；回归测试已加入；学生能解释 bug 原因。

11.5 练习 5：写 Snakemake Rule

目标：让 Codex 把已有 CLI 命令包装成 Snakemake rule。

输入：已有 CLI 命令 + 输入输出文件列表

Codex prompt：

We are in Month 2 of an AI-first LQCD meson DA analysis-agent project.

Month 1 produced a config-driven Python CLI.

Task:

Create `workflow/Snakefile` and `workflow/rules/\*.smk`.

Requirements:

- Use `configs/l64c64a076\_m140.yaml` as main config
- Call existing Python CLI commands; do NOT put analysis logic in Snakefile
- Produce at least: audit_report.json, result.json, summary.csv, case_manifest.json
- Add dry-run instruction to `docs/month2\_demo.md`
- Add pytest smoke test using `snakemake --dry-run` on tiny config

Constraints:

- Do not change analysis algorithms
- Do not hard-code absolute paths
- Do not overwrite raw data

禁止事项：不允许在 Snakefile 中写复杂分析逻辑。

验收标准：`snakemake --dry-run` 展示正确依赖图；真实运行生成预期输出。

11.6 练习 6：写 Fit Diagnostics

目标：让 Codex 实现 fit stability metrics。

输入：summary.csv 的字段定义 + 诊断需求

Codex prompt：

Task:

Implement Fit Strategy Postdoc diagnostics in:

`src/lqcd\_agent/analysis/fit\_strategy\_postdoc.py`

Requirements:

- Read summary.csv with columns: case_id, model, tmin, tmax, param, value, error, chi2_dof, status
- Compute diagnostics:
 1. chi2 quality (flag if chi2/dof > acceptable threshold)
 2. tmin stability (check if neighboring tmin results differ by >1 sigma)
 3. model dependence (1-state vs 2-state sigma difference)
 4. fit failure rate
- Write fit_strategy_plan.json with 2-5 recommended hypotheses
- Each hypothesis must have a physics reason (not just "lowest chi2")
- Do NOT output a brute-force grid scan of all possible tmin combinations

Add tests for stable case, large chi2 case, and model-dependence case.

禁止事项: 不允许做 grid scan; 不允许不写物理理由就推荐 fit。

验收标准: 生成的 hypotheses 有物理理由; 每个 case 2-5 个候选; 测试覆盖 3 种情况。

11.7 练习 7: 写 Critical Referee Objection Logic

目标: 让 Codex 实现反对意见检查规则。

输入: fit_strategy_plan.json + summary.csv 的字段

Codex prompt:

Task:

Implement Critical Referee checks in:

`src/lqcd\_agent/critical\_referee/critical\_referee.py`

The referee must find problems, not give compliments. Implement at least 10 check rules:

1. False plateau detection (plateau has <3 stable points)
2. Fit window cherry-picking (selected tmin is suspiciously narrow)
3. Overfitting check (2-state has many more params but similar chi2/dof to 1-state)
4. Prior dominance check (posterior $\sim$ prior within 0.5 sigma)
5. Tmin sensitivity check (E0 shifts > 2 sigma across tmin values)
6. Pz-dependence anomaly (trend reverses after normalization)
7. Z-dependence non-smoothness (oscillations in z-space)
8. Local ME normalization violation (z=0 value deviates from 1 by >3 sigma)
9. Real/imag symmetry violation
10. Conclusion overreach (claim stronger than data supports)

Output: critical_referee_report.json with major_objections[] and minor_objections[]. Each objection must have: issue type, claim, evidence (specific numbers), requested_check.

Add tests for good case (0 major objections), ambiguous case (1-2 major objections), and bad case (≥ 3 major objections or recommend reject).

禁止事项: 不允许只输出“看起来不错”的结论；不允许没有 evidence 的 objection。

验收标准: good case 上 0 major objection；ambiguous case 上至少 1 个有数字 evidence 的 objection。

11.8 练习 8: 写 Approval Packet Generator

目标: 让 Codex 实现 approval packet 生成。

输入: decision_log.json + agent 报告路径列表

Codex prompt:

Task:

Implement approval packet generator in:

`src/lqcd\_agent/graph/approval\_packet.py`

When Analysis Coordinator determines high risk, generate this directory:

outputs/<case_id>/approval_packet/

00_requested_decision.md	# specific question for human
01_summary_for_human.md	# human-readable summary with key numbers
03_fit_comparison.csv	# copy from outputs
04_critical_referee_report.json	# copy from outputs
05_ai_recommendation.json	# AI's preferred choice + reason
06_decision_options.yaml	# structured options with risk_if_chosen
07_human_decision.yaml	# template for human to fill

The 06_decision_options.yaml must include:

- case_id, risk_level, question
- ai_recommendation with preferred option and reason
- 2+ options, each with label, risk_if_chosen, and ai_preference
- deadline
- fallback_if_no_response: "hold_and_remind"

Add test that generates a complete approval packet from mock data.

禁止事项: 不允许 hard-code 具体 case 信息；不允许生成空文件。

验收标准: 能从 mock decision_log 生成完整 approval packet；06_decision_options.yaml 格式正确。

11.9 练习 9: 写 Memory Update Function

目标: 让 Codex 实现结构化 case memory 更新。

输入: memory entry schema + decision_log.json

Codex prompt:

```
Task:
Implement case memory update in:
`src/lqcd_agent/memory/store.py`

Requirements:
- Read decision_log.json and extract memory-relevant fields
- Update accepted_cases.yaml when risk=low AND status=success
- Update pending_cases.yaml when risk=medium
- Update rejected_cases.yaml when status=failed or human rejected
- Each entry must include: case_id, ensemble, selected_fit, risk_level,
  human_approved, key_reasons, output_paths
- Support query by case_id
- Support list by risk_level

Add tests for accepted, pending, and rejected memory updates.
Keep all memory files human-readable YAML.
```

禁止事项: 不允许覆盖已有 case (除非明确 append/update); 不允许使用数据库 (Month 3 阶段用 YAML)。

验收标准: 三类 cases 正确分文件保存; 能按 case_id 检索; 能按 risk_level 列出。

11.10 练习 10: 写 Final Report Template

目标: 让 Codex 生成 human-readable Markdown 报告。

输入: 一个 case 的全部 agent 输出

Codex prompt:

```
Task:
Implement final report generator in:
`src/lqcd_agent/report/generate_report.py`

The report should read all agent outputs and produce a structured Markdown report:

# Analysis Report: <case_id>

## 1. Case Metadata
(config path, git commit, timestamp, ensemble info)

## 2. Data Audit Summary
```

```

## 3. Correlator Diagnostics
(key findings from correlator_diagnostics.json)

## 4. Fit Strategy
(recommended hypotheses from fit_strategy_plan.json)

## 5. Matrix Element Diagnostics
(key findings from matrix_element_diagnostics.json)

## 6. Critical Referee Review
(major and minor objections)

## 7. Analysis Coordinator Decision
(recommended fit, risk level, reasons)

## 8. Human Approval Status
(if applicable)

## 9. Next Actions

## 10. Reproducibility
(exact command, config path, git commit)

Add test that generates a report from mock data.

```

禁止事项：不允许硬编码 case 信息；不允许跳过任何 section。

验收标准：生成的报告包含全部 10 个 section；每个 section 都有实质性内容（不是 placeholder）。

11.11 Codex 使用黄金法则

1. **每次只让 Codex 做一个小任务。**不是”帮我做完整项目”，而是”生成 Pydantic schema for config”。
2. **必须提供输入和输出示例。**Codex 不是读心术师。
3. **必须让 Codex 写测试。**不写测试的 Codex prompt 是不完整的。
4. **必须自己运行和检查。**Codex 生成的代码可能看起来对但运行时出错。
5. **用 traceback 驱动修复。**把错误信息完整给 Codex，不要只说”修一下”。
6. **每次完成一个小任务就 commit。**不要把一周所有修改堆到一个 commit。
7. **不要盲目信任 Codex 的物理解释。**物理判断最终由人类负责。

本章交付清单：

- 完成至少 8/10 个练习；

- 维护 Codex 使用日志（每次记录：目标、prompt 摘要、是否成功、手动检查了什么）。

第七部分 最终验收与升级路线

12 Final Demo 验收标准

12.1 最终产品必须展示的能力

三个月结束时，学生必须在一次 15–20 分钟的 demo 中展示以下 9 项：

1. 一个 **golden example case** 的完整运行：从 config 到 final report 的全流程。
2. 一条完整 **pipeline**：CLI + Snakemake workflow + agent graph。
3. 一个 **multi-agent decision flow**：每个 node 的输入输出清晰可追踪。
4. 一份 **decision_log**：包含推荐 fit、risk level、理由和 evidence。
5. 一个 **critical objection**：Critical Referee 提出的 major objection，含具体数字 evidence。
6. 一次 **targeted rerun suggestion**：系统发现问题后给出的结构化 rerun 建议。
7. 一个 **human approval packet**：high risk case 生成的完整审批包（7 个文件）。
8. 一个 **memory update**：accepted / rejected / pending case 被正确记录。
9. 一份 **final report**：人类可读的分析报告（含 10 个 section）。

12.2 导师验收问题清单

Demo 进行时，导师应提问以下问题：

1. **复现性**：这个结果怎么复现？需要什么命令、config 和数据？
2. **AI 决策**：AI 推荐了什么？为什么推荐这个？
3. **反对意见**：Critical Referee 提出了什么反对意见？有没有数字 evidence？
4. **风险评估**：这个 case 的风险等级是什么？为什么？
5. **人类介入**：是否需要人类审批？什么时候触发？如果不审批会怎样？
6. **依赖管理**：如果改一个 config 参数，哪些 step 会重跑？
7. **失败诊断**：如果 fit 失败，系统会如何诊断和报告？
8. **经验积累**：这个 case 如何进入 memory？下次如何检索？
9. **系统边界**：当前系统还不能做什么？下一步要做什么？

12.3 每月汇报模板

Month X Report

1. This month's goal


```
2. Implemented features
3. Reproduction commands
4. Main outputs
5. Tests (pytest result: pass/fail; important tests)
6. Codex usage (prompts used, code reviewed, bugs fixed)
7. Scientific checks (fit stability, sanity checks, referee flags)
8. Failures and lessons
9. Questions for advisor
10. Plan for next month
```

12.4 导师检查清单（三个月汇总）

Month 1 检查重点：

- 是否能复现 golden example? config 是否完整?
- result schema 是否足够结构化?
- pytest 是否真正保护关键行为?
- 学生是否理解 basic analysis outputs?

Month 2 检查重点：

- Snakemake 是否正确管理依赖?
- Analysis Engineer 是否安全? Analysis Coordinator 是否记录理由?
- Critical Referee 是否能指出真实风险?
- decision log 是否可读、可追责?

Month 3 检查重点：

- Agent state 是否清楚? agent node 是否有明确输入输出?
- human approval 是否在正确风险节点被触发（而非每个节点）?
- targeted rerun 是否真的闭环?
- memory 是否保存结构化经验?
- 学生是否能解释系统的边界和风险?

12.5 最终交付清单

1. GitHub repo (含 README、pyproject.toml、完整目录结构)
2. golden example reproduction command
3. Snakemake workflow
4. LangGraph 或 Python state-machine prototype

5. pytest test suite (15+ tests)
6. decision logs (至少 3 个 cases)
7. memory examples (accepted/rejected/pending)
8. evaluation cases (good/ambiguous/bad)
9. final demo report
10. 15-20 分钟 presentation slides

13 未来升级方向：如何变成更高级的系统

三个月后，如果原型成功，以下是逐步升级的方向。核心理念不变：**AI agents 逐步承担更多 routine labor 和 analysis decisions；人类保留 high-value decision making 和 final accountability。**

13.1 升级路线图总览

升级方向	从（Month 3 状态）	到（长期目标）
Analysis Coordinator	rule-based	LLM-assisted reasoning + guardrail
Critical Referee	single referee	multi-agent debate (proponent vs alternative vs referee)
Memory	YAML / JSON	SQLite / DuckDB / vector database / RAG
Agent 规模	single case	ensemble-level campaign
LQCD Physics Expert	basic rule-based checks	LLM-based LQCD physics reasoning
Perturbative Matching Expert	暂不实现	quasi-DA → light-cone DA 全流程
Observability	run.log + decision_log	MLflow / OpenTelemetry tracing + dashboard
Human Approval	file-based approval packet	interactive review UI + notification
AI Development	Codex-assisted coding	semi-autonomous research engineering loop
Physics goal	golden / fake example	real production DA analysis

13.2 升级 1：从 Rule-Based 到 LLM-Assisted Analysis Coordinator

第一版 Analysis Coordinator 使用规则是正确选择（透明、可测试、可控）。下一步引入 LLM：

- LLM 读取 structured diagnostics，生成 human-readable reasoning
- LLM 提出 candidate explanations 和 targeted rerun plan
- Rule-based guardrail 检查 LLM 输出是否符合 schema 和安全规则
- 中高风险结论仍需 human approval

13.3 升级 2: 从 Single Referee 到 Multi-Agent Debate

引入多个 reviewer 角色, 模拟真实合作组的内部讨论:

- Fit Strategy Postdoc A (proponent): 提出主方案
- Fit Strategy Postdoc B (alternative): 提出备选方案
- Critical Referee: 寻找不稳定点和漏洞
- Lattice QCD Physics Expert: 检查整体物理趋势
- Perturbative Matching Expert: 检查 matching/factorization 相关系统误差
- Analysis Coordinator: 综合各方意见, 做出决策

每个 reviewer 必须输出结构化 JSON (含 blocking_issues、non_blocking_warnings、recommended_action、confidence、evidence_files), 以保证 debate 可测试、可记录、可复现。

13.4 升级 3: 从 YAML Memory 到 SQLite/DuckDB + RAG

当 case 数量超过几十个时, YAML 检索不便。升级路径:

- SQLite / DuckDB 存储 structured case records
- 支持按 ensemble、momentum、risk level、failure pattern 检索
- RAG 让 agent 从历史 cases 中检索相似情况和 prior decisions
- 自动生成 failure pattern summary 和常见 risk 统计

13.5 升级 4: 从 Single Case 到 Ensemble-Level Campaign

扩展到多个 ensemble、多个 momenta、多个 z values:

- 自动生成 campaign config
- Snakemake 批量调度所有 cases
- Campaign-level Analysis Coordinator 检查 ensemble dependence、 P_z dependence、 z -dependence
- 生成 campaign summary report

13.6 升级 5 & 6: 加入 LQCD Physics Expert 和 Perturbative Matching Expert

从 Month 3 的 basic rule-based checks 升级为完整的 LLM-based 物理审查:

- Lattice QCD Physics Expert: 检查 discretization effects、finite-volume effects、excited-state contamination、ensemble consistency
- Perturbative Matching Expert: 检查 matching kernel、renormalization scheme、factorization scale、endpoint behavior

13.7 升级 7 & 8: Observability + Interactive Review

- MLflow 或 OpenTelemetry tracing 记录每个 node 的耗时、LLM 调用成本、失败率
- Dashboard 展示 analysis DAG、agent graph、decision log、plots
- Human Approval 从 file-based 升级为交互式 Web UI (通知 + 一键审批 + 批注)

13.8 最终理想形态: LQCD Analysis Subgroup as Code

最终目标不是做一个“更聪明的 chatbot”，而是把整个 LQCD meson DA 分析流程变成一个可审计、可复现、可学习的 AI analysis subgroup。

在这个理想系统中：

导师提出物理目标（例如“在 ensemble A 上分析 pion DA，检查 fit stability、bare ME systematics 和 matching uncertainty”），系统自动组织分析流程、运行计算、检查质量、发现异常、提出少量高价值判断和 rerun 建议，并把所有代码、数据、图、日志、决策和人工审批保存下来。AI agents 自动承担大部分 routine labor 和越来越多的 routine analysis decisions；人类主要提出 physics goal、检查关键 evidence、审批高风险结论，并对最终科学结论负责。

这就是本项目的核心愿景：用 AI agents 构建一个像真实 LQCD 分析子组一样工作的科研系统，把人类从重复劳动中解放出来，让人类专注在只有人类能做的判断上。

本章交付清单：

- 能说出至少 5 个未来升级方向；
- 能解释从 rule-based 到 LLM-assisted 的渐进策略；
- 能描述“LQCD Analysis Subgroup as Code”的理想形态。

附录 A：推荐最终 Repo 结构

```

lqcd-meson-da-agent/
  README.md
  pyproject.toml
  configs/
    164c64a076_m140.yaml
  examples/
    164c64a076_m140/
  src/
    lqcd_agent/
      schema/                                # Pydantic schemas
      analysis/                              # Analysis core (deterministic)
        correlator_diagnostics.py
        fit_strategy_postdoc.py
        matrix_element_analyst.py
      audit/                                # Data integrity checks
      cli/                                  # CLI entry points
      analysis_engineer/                    # Analysis Engineer wrapper
      critical_referee/                     # Critical Referee checks
      analysis_coordinator/                 # Analysis Coordinator / PI
      graph/                               # State machine / LangGraph
        state.py
        build_graph.py
        run_graph.py
        approval_packet.py
      report/                              # Report generator
      memory/                              # Case memory
        schema.py
        store.py
    workflow/
      Snakefile
      config.yaml
      rules/
    tests/
    evaluation/
      cases/
      expected/
      run_evaluation.py
    memory/
      accepted_cases.yaml
      rejected_cases.yaml
      pending_cases.yaml
    docs/

```

```
outputs/  
.gitkeep
```

附录 B：参考资料与工具选择依据

- Snakemake 适合 reproducible and scalable scientific data analysis，尤其适合文件依赖清楚的科研 workflow。
- LangGraph 适合 stateful agent workflow、durable execution 和 human intervention interrupt / resume。
- TradingAgents 类系统体现了 specialized roles、debate、risk management 和 final decision gate 的 multi-agent 设计思想。
- Dagster 的 software-defined assets 和 Prefect 的 Python-native retry / observability 很有价值，但本项目第一主线更适合 Snakemake + LangGraph 的组合。
- Nextflow 在容器化、跨平台、标准化大 pipeline 中很强，但对当前 Python-heavy、LQCD exploratory analysis 来说，Snakemake 更适合。

Online References

- TradingAgents project: <https://tradingagents-ai.github.io/>
- TradingAgents paper: <https://arxiv.org/abs/2412.20138>
- Snakemake documentation: <https://snakemake.readthedocs.io/>
- LangGraph durable execution: <https://docs.langchain.com/oss/python/langgraph/durable-execution>
- Nextflow documentation: <https://www.nextflow.io/docs/latest/>
- Prefect documentation: <https://www.prefect.io/how-it-works>
- Dagster software-defined assets: <https://docs.dagster.io/>

给学生的最后提醒

本项目的目标不是让 AI 只做辅助性的零碎工作，而是让 AI agents 尽可能承担科研工程里的重复劳动、流程执行、初步检查和越来越多的 analysis decisions。人类只在关键决策、高风险审批和最终科学责任节点介入。

你三个月后最重要的能力不是“会调用某个工具”，而是知道如何把一个真实科研分析问题拆成一个**尽量自动化、可复现、可检查、可审计、只在必要时才需要人类介入的 AI agent system**。

最终的目标是：**你成为一个科研 AI agent 搭建初级专家**——知道如何拆任务、如何让 AI 自动化执行、如何测试、如何管理 workflow、如何设计 agent state、如何记录决策、如何在关键节点保留人工审查，以及如何向导师展示一个可靠的初级系统。